# Local Decidability as a Condition for Scalable Agentic Coding

Mark Farkas
*artalis.io*

***Abstract*—In projects examined here, coding agents increased implementation throughput, but code production was only one constraint on system quality. Four cases cover a networking substrate, a converging capability-secure runtime, an implemented inference engine and proposed successor architecture, and an experimental LP and MIP solver. The cases are consistent with the hypothesis that agentic coding scales when architecture converts system-level obligations into locally decidable tasks whose results are observable and compositionally meaningful. The practical dividing line is not whether agents can implement components, but whether component-level correctness composes into observable whole-system quality. The engineering value of agent assistance varies with reasoning scope, invariants, feedback, coupling, and review burden. Architectural judgment defines boundaries and validation regimes, but expert attention is limited and cannot replace empirical evidence. Agent assistance amplifies its architecture and feedback regime; implementation throughput does not reduce the cost of managing complexity.**

***Index Terms*—Local decidability, agentic coding, AI-assisted programming, systems engineering, software architecture, compositional correctness, system-level validation**

## I. Introduction

Over approximately six months beginning in late February 2026, I used coding agents to implement and review several systems projects, all built from scratch during the study period. The agents produced the source code, tests, documentation, build integration, and platform adaptations; I did not write source code manually. My role was to define architecture, decompose work, specify invariants, interpret results, and accept or reject changes. This arrangement substantially increased the amount of implementation one engineer could attempt, but it did not reduce the corresponding need for architectural judgment and system-level validation.

Claude Code served as the primary coding system, using successive Claude Opus 4.6, 4.7, and 4.8 models as they became available. OpenAI Codex, using GPT-5-series models through GPT-5.6, reviewed every change and acted as the principal scrutinizer. Open-weight Qwen3.6 27B and Qwen3.8 27B models supported additional review, testing, and smaller tasks. The toolchain therefore changed during the study period; this was an evolving practitioner experiment rather than a controlled comparison of fixed models.

The human contribution was not neutral supervision. I brought approximately 15 years of experience in systems architecture, systems engineering, operations research, and engineering leadership. This background shaped the decomposition, invariants, acceptance criteria, and interventions described here. It also limits generalizability: the results should not be interpreted as what coding agents achieve without experienced architectural guidance.

Here, agentic coding denotes a repository-level workflow in which generative models inspect existing code, modify multiple files, build the system, execute tests and benchmarks, diagnose failures, and revise their output in response to observed results. Within assigned tasks, the agents performed most command execution autonomously; I seldom ran commands myself. The workflow was nevertheless human-directed through architectural decisions, task decomposition, acceptance criteria, and intervention.

The central claim is that increased implementation throughput does not proportionally reduce the cost of managing system complexity. That complexity lies in preserving coherent behavior across components, platforms, states, failure modes, and performance regimes. It includes selecting abstractions, assigning responsibilities, stating invariants, detecting cross-layer regressions, and rejecting locally plausible changes that weaken the system as a whole.

**AI makes code cheap. It does not make complexity cheap.**

The cases provide empirical counterevidence to the claim that automating source-code production makes software construction a solved problem. Source-code generation was nearly fully automated, yet decomposition, invariant design, integration review, diagnosis, and system-level validation remained binding constraints. Coding may be approaching automation; systems engineering is not.

The argument draws on four projects. Keel is a portable asynchronous networking and transport substrate. Hull is a capability-secure execution substrate built above Keel. Sail, currently developed in the repository named bitnet.c, is a compact inference engine and planned library architecture. Ralph is an experimental LP and MIP solver. Keel provides retrospective positive evidence; Hull provides evidence from a converging architecture; Sail provides a prospective architectural test grounded in an implemented system; and Ralph provides a negative experimental result on harder problem instances, where locally valid algorithmic changes poorly predicted whole-solver convergence, numerical robustness, and performance.

To compare these differences, the paper introduces a qualitative scale of engineering complexity based on reasoning scope, coupling, compositionality, feedback quality, and interaction growth.

Existing evaluations measure repository-level task completion and autonomous task horizons, but do not directly characterize when locally correct changes compose into system-level quality [11, 12].

Recent work examines runtime-structured task decomposition and benchmarks model performance on software-architecture tasks [13, 14]; this paper instead asks when system architecture makes agent-produced changes locally assessable and predictive of whole-system quality. Recent audits also report increased coupling and structural degradation in agent-generated systems [15]; the present framework interprets such degradation as especially consequential when local changes are weakly observable or non-compositional.

This paper proposes local decidability as the central explanatory variable linking increased agent-assisted implementation throughput to scalable engineering throughput.

These longitudinal practitioner cases are not controlled experiments. They differ in domain, maturity, success criteria, model generation, and available feedback, and no counterfactual implementations without coding agents exist. The evidence consists of repository structure, normative contracts, executable tests, and dated benchmark records; cross-case conclusions are qualitative and do not compare model productivity quantitatively. The paper therefore does not estimate the causal effect of agent use. It develops a systems-engineering hypothesis from observed contrasts and derives implications that can be tested in future comparative studies.

## II. Analytical framework

System quality is not proportional to code volume. The relevant question is when increased agent-assisted implementation throughput creates engineering value. As an analytical lens, the engineering value of agent assistance tends to increase with implementation throughput, local decidability, and observability, and to decrease with cross-component coupling and review burden. These factors interact and are not assumed to be independent; the relation is a qualitative dependency statement, not a mathematical model. Domain knowledge, available engineering time, problem difficulty, project maturity, oracle quality, and computational budget also affect outcomes.

A further hypothesis concerns scale. When agent-generated systems grow without deliberate decomposition and expert architectural guidance, cross-component assumptions and review burden can grow faster than enforceable invariants and observable evidence. The resulting gap creates verification debt: accumulated assurance work produced when changes and dependencies arrive faster than contracts, tests, and representative evidence can validate them. Beyond some scale, this debt imposes a practical limit on reliable progress.

### A. Local decidability and decomposition

Local decidability is the degree to which the correctness of a change can be determined from a bounded part of the system. Two forms matter. Implementation-local decidability asks whether a component satisfies its interface or contract. Outcome-local decidability asks whether that local correctness predicts the claimed system-level outcome. Source-code size does not determine reasoning scope: a short function may have globally coupled semantics, whereas a large provider implementation may be locally decidable if it realizes a complete, testable contract. The paper's central hypothesis is:

> *Agentic coding scales when architecture converts global obligations into bounded, locally decidable tasks whose correctness is observable and compositionally meaningful.*

Effective decomposition assigns distinct semantic responsibilities to components and bounds the context required to assess a change. A boundary is effective when it separates reasons for change, makes ownership unambiguous, and prevents orthogonal concerns from collapsing into shared conditional logic. Decomposition is not synonymous with increasing module count; excessive fragmentation may merely relocate complexity into coordination. The relevant question is whether each boundary corresponds to a genuine semantic distinction with separately stateable obligations: protocol versus transport, policy versus mechanism, immutable model state versus request-local state, or LP relaxation versus tree search.

Local decidability can be increased by stating dependency rules explicitly, separating orthogonal dimensions, assigning each responsibility a single owner, and requiring each new feature to have a natural architectural home. Warning signs include changes that require parallel conditionals across unrelated files and interfaces that expose provider-specific details to otherwise neutral code.

### B. Invariants and interfaces

Interfaces delimit the assumptions that components may make about one another. Invariants state the properties that must remain true across success, failure, cancellation, and teardown. Across the cases examined here, relevant invariants include exactly-once terminal callbacks, explicit buffer ownership, bounded queues, immutable shared state, request-local mutation, auditable authority flow, and deterministic fallback behavior.

Invariants become enforceable through normative contracts, narrow data types, capability-bearing handles, one-directional dependency rules, explicit lifecycle states, and conformance tests that exercise the same semantics across providers. An invariant recorded only in a prompt or an engineer's memory remains weak. Assurance is strengthened when the invariant is stated in a normative contract, encoded in language mechanisms and APIs, checked automatically, and, where warranted, subjected to formal verification.

### C. System-level feedback

System-level feedback is informative when it is timely, attributable, representative, and repeatable, and when consistent evaluation criteria are applied at the level of the claimed outcome. Unit tests provide dense local feedback, but cannot by themselves establish end-to-end properties. Accordingly, validating a networking substrate requires cross-backend conformance and lifecycle tests. Validating an inference system requires token parity, numerical coherence, memory-safety checks, and end-to-end performance measurements. Validating an LP and MIP solver requires broad instance suites,

TABLE I
ENGINEERING COMPLEXITY REGIMES.

| Regime | Defining attributes | Typical agent value | Expert value and best complement | Cases |
|---|---|---|---|---|
| 0) Reproduced microtask | Familiar pattern, tiny state space, negligible coupling, immediate oracle. | High apparent implementation speed; weak evidence of systems capability. | Low incremental value. General software engineer or technically competent domain user. | TODO and isolated coding demonstrations. |
| 1) Bounded conventional application | Established framework boundaries, modest state, direct acceptance tests, attributable failures. | High throughput with bounded review and integration effort. | Moderate value. Product-oriented domain engineer with framework and operations experience. | Simple CRUD or web applications. |
| 2) Contract-compositional system | Explicit semantic boundaries and enforceable invariants; local conformance predicts substantial system behavior. | Very high when expert architecture creates stable, bounded obligations. | High, strongly leveraged value. Systems architect with domain, concurrency, lifecycle, and platform expertise. | Keel, Hull. |
| 3) Matrix-coupled system | Individually tractable dimensions form a large support and validation cross-product. | High within one matrix cell; lower and more costly across integrations. | Very high value. Performance systems engineer with numerical, hardware, and model-format expertise. | Sail. |
| 4) Emergent-outcome system | Local validity weakly predicts global quality; feedback is delayed, noisy, and distribution-dependent. | Useful for implementation and experiments; throughput weakly predicts quality. | Indispensable but bounded value. Operations-research or numerical-optimization expert skilled in experimental design. | Ralph. |
| 5) Open-world socio-technical system | Outcomes depend on changing environments, adversaries, operators, institutions, and rare interactions. | Conditional and component-specific; implementation throughput cannot provide complete assurance. | Collective and continuous value. Multidisciplinary teams spanning architecture, safety, security, operations, human factors, and regulation. | Autonomous transport, air-traffic management, global cloud control planes, critical infrastructure. |

objective and feasibility checks, numerical diagnostics, and whole-search performance measurements.

Observability can be improved through trusted reference implementations, differential testing, representative benchmark suites, sanitizer and property checks, deterministic traces, performance gates, and explicit failure taxonomies. Feedback density also matters: short causal loops make it easier to associate a change with its consequences. Delayed or noisy feedback encourages optimization against incomplete or misleading proxies.

### D. Engineering complexity regimes

The following scale classifies engineering complexity by the scope of evidence required to assess a change, not by source-code volume or computational complexity. Its defining attributes are reasoning scope, cross-component coupling, compositionality of local guarantees, oracle quality, and growth of interactions as features are added. The regimes are analytical categories rather than fixed properties: architecture can move a project toward or away from local decidability, although it cannot remove difficulty intrinsic to the domain. The scale predicts differences in the conditions under which additional implementation throughput yields engineering value; it does not rank projects by importance, source-code size, or absolute difficulty.

The first two regimes are analytical baselines and are outside this study. Reproduced microtasks exercise familiar patterns with immediate oracles, while bounded applications largely require finite specification and integration effort from a domain engineer. Repository-level benchmarks and task-horizon studies likewise distinguish bounded issue resolution from longer, messier work, where success decreases as coordination and context grow [11, 12]. Such tasks are poor tests of the present hypothesis because their reasoning scope, coupling, and validation cost are limited by construction.

Expert value changes form across the scale. At lower regimes it consists mainly of specification and acceptance. In contract-compositional systems, expertise creates the boundaries that make agent assistance scalable. In matrix-coupled and emergent systems, experts must also constrain interaction growth and design representative evidence, yet their judgment cannot reliably predict every system-level outcome. Open-world systems require continuing collective assurance rather than a single complementary expert.

## III. CASE ASSESSMENTS

### A. Keel: retrospective positive evidence

Keel is a portable asynchronous networking and transport substrate [1, 2]. Its architecture separates three major concerns: protocol semantics, event execution, and platform-specific transport provision. Readiness-based and completion-based backends differ in how operations are initiated and completed, while protocol implementations consume a stable Keel-level contract. Provider interfaces mediate transport operations and prevent protocol implementations from depending directly on POSIX semantics. The repository contains separate provider integrations for POSIX, Winsock, lwIP, and UEFI environments.

The decomposition is semantic rather than cosmetic. Streams, datagrams, and listeners have distinct communication and ownership semantics but share explicit lifecycle disciplines. For example, readiness-based and completion-based backends use different operation mechanics, yet both can be tested against the same terminal-outcome contract. Backend work is therefore locally decidable within the contract, without simultaneous reasoning about HTTP parsing, TLS policy, or the internal mechanics of other event engines. Normative contracts specify ownership, cancellation, backpressure, synchronous completion, and confirmed-detachment semantics [2].

Keel exhibits comparatively high implementation-local decidability because provider mechanics and protocol semantics have distinct architectural homes. These boundaries are reinforced by explicit state machines and contract tests for terminal-outcome behavior and absence of silent truncation. Its feedback is comparatively dense: unit tests, backend conformance suites, sanitizer checks, and integration tests can expose failures close to their originating boundary.

I defined and revised the decomposition, rejected boundary violations, and identified invariants that required executable tests. The coding systems accelerated implementation, test construction, documentation, and the mechanical propagation of contract changes across backends. Once the boundaries were stable, many tasks became locally decidable: the architecture reduced the context required to implement and assess a change.

This local decidability remains incomplete: a change that appears locally consistent can still violate temporal behavior spanning cancellation, callback reentrancy, resource ownership, and backend-specific completion semantics. Expert review cannot establish correctness by itself. Architectural judgment identifies which properties require assurance and which feedback mechanisms can expose violations.

### *B. Hull: evidence from a converging architecture*

Hull is an execution substrate organized around capability-based authority for native and agentic applications [3]. Its central division is between mechanism and policy. A deliberately small native core enforces access to resources, while higher-level layers express application behavior and policy where appropriate. The core is reinforced by lower-level protection mechanisms, including W^X memory policy, read-only sealed dispatch tables, and operating-system sandboxing through pledge and unveil on OpenBSD, seccomp and Landlock on Linux, and Seatbelt on macOS. Corresponding Windows enforcement remains under development. Authority is conveyed through explicitly granted capabilities rather than ambient access. Hull contains a broader set of subsystems than can be assessed here. Illustrative boundaries separate lexical path processing, filesystem resolution and operations, blob storage, process execution, networking, and application services because these responsibilities impose distinct naming, authority, and lifetime semantics.

Hull uses Keel as the networking and transport substrate beneath HTTP and other native services. This reuse keeps event-loop and transport semantics outside Hull's authority model, reducing duplicated mechanisms and cross-layer assumptions. Static composition further constrains which mechanisms and authorities are present in a deployed application: build-time selection makes authority and dependency flows more auditable than unrestricted runtime discovery.

Hull is less mature than Keel, so I treat it as ongoing rather than retrospective evidence. Evidence of architectural convergence is strongest where each component has one security responsibility, such as lexical path normalization, policy-constrained resolution, operation on an authorized object, or exposure of a higher-level service. The architecture also encodes negative authority: a component without the required capability should have no alternate path to a protected resource. Manifests, capability checks, operating-system sandboxing, static linkage, and audit paths reinforce this constraint at complementary layers. Unit, fuzz, and application-level tests, together with security review, provide partial evidence, but Hull's validation record remains less mature than Keel's cross-backend conformance evidence.

Architectural judgment in Hull includes separating convenience from enforcement even when combining them would simplify a local implementation. In practice, this means rejecting helpers that bypass authority boundaries, placing policy at the appropriate layer, and keeping the native enforcement core small enough to audit.

This architectural judgment also has limits. Capability systems can fail through omitted restrictions and unsafe composition even when individual checks appear correct. An engineer may define an apparently clean authority boundary yet overlook an indirect channel, an unsafe object lifetime, or a hazardous interaction among individually reasonable capabilities. Review by additional models can broaden the search for defects, but agreement among models is not independent evidence when they may share training-derived assumptions and blind spots. Security claims still require explicit threat models, adversarial testing, implementation review, and enforcement or formal analysis appropriate to the claimed boundary.

### *C. Sail: a prospective architectural test*

Sail, an implemented inference engine and proposed successor architecture, exposes the combinatorial complexity of supporting multiple model, representation, execution, and hardware dimensions [4]. Its support matrix spans model families, tensor roles and representations, quantization formats, compute backends, execution phases, memory layouts, tokenizer behavior, and sampling behavior. Each dimension can often be reasoned about in isolation, but their cross-product creates integration obligations that cannot be validated independently. Subtle differences in numerical representation, accumulation order, and conversion behavior can remain locally plausible while changing end-to-end numerical results. Supporting a new model family may require changes to tensor identification, normalization, attention, mixture-of-experts routing, or state-space operations. Adding a quantization format can introduce distinct storage layouts, conversion rules, and backend kernels. Backend capabilities may cover only subsets of operations, numerical representations, or fused execution paths. Prefill and single-token generation present different computational shapes and performance constraints. Additional techniques introduce further architectural axes: multi-token prediction changes output structure and decoding control, DFlash adds block-diffusion speculative drafting, and TurboQuant changes KV-cache representation and the associated attention path [8-10]. Agreement on selected logits is insufficient to establish end-to-end equivalence if tokenization, sampling, cache updates, or decoding control diverge.

The current implementation addresses this cross-product through explicit responsibility and state ownership. A dedicated model-architecture layer owns model-family-specific rules. A quantization registry centralizes format metadata and

declared backend capabilities. Backend model objects own immutable backend-resident state, while session objects own request-local mutable state. Execution planning remains separate from backend execution (portable scalar CPU, AVX2, AVX512, ARM NEON128, CUDA, and Metal), while backend-specific kernels and shader identities remain below a backend-neutral operation layer. Tokenization and sampling remain first-class architectural responsibilities rather than incidental utilities.

Despite its backend breadth, Sail deliberately constrains the supported cross-product to a small set of understandable quantization formats and important model families. Design goals include minimal external dependencies, a small auditable implementation, exact token behavior, and prefill and generation performance comparable to llama.cpp. Sail therefore provides a prospective architectural test rather than a completed outcome: the current implementation supplies observations that motivate the successor decomposition but does not establish its stated targets.

In Sail, local decidability requires assigning each dimension a single architectural owner and making cross-dimensional choices explicit in an execution plan. Relevant invariants include immutable model structure, request-local session state, table-driven quantization capabilities, deterministic placement and fallback, and exact tokenization and sampling contracts. Validation must combine boundary tests, differential comparisons with trusted reference implementations, cross-backend numerical-coherence checks, exact token-sequence checks, sanitizers, memory accounting, and phase-specific performance benchmarks.

The engineering value of agent assistance is comparatively high for constrained tasks, such as implementing one kernel against a scalar reference or propagating a new capability through a stable registry. That value decreases when success depends on several interacting dimensions and an apparently faster path changes layout, fallback, or token behavior without detection. Architectural judgment can constrain the active support cross-product but cannot eliminate interactions among the retained dimensions. Scope discipline is therefore a technical control, not merely a project-management preference.

### *D. Ralph: successful small cases, limited hard-case convergence*

Ralph was an experimental dependency-free LP and MIP solver used partly to test the limits of agentic coding. The dependency constraint kept numerical linear algebra, including sparse LU factorization and basis updates, stability management, and recovery behavior inside the system under study rather than delegating them to mature numerical libraries. It nevertheless solved many smaller NETLIB LP instances and multiple other LP and MIP models to optimality. The repository implements presolve, scaling, primal and dual simplex components, sparse factorization and update machinery, cut generation, branching, and benchmark infrastructure, including comparisons with GLPK and NETLIB-based gates [5]. The comparator, GLPK, is a mature optimization system providing primal and dual simplex, interior-point, and branch-and-cut facilities [6]. On smaller or easier instances, Ralph often reached correct optima with competitive runtimes. In selected specialized cases, including instances handled by its network-simplex path and other LP or MIP models with favorable substructure, recorded runs were faster than GLPK. These observations establish narrow areas of competence, not general performance superiority. Ralph's negative experimental result is not categorical failure; rather, reliable convergence was not achieved as instances became larger, harder, and more numerically or algorithmically coupled.

A dated stress diagnostic provides concrete evidence for this hard-case limitation. On 2026-03-01, the repository recorded 72 solver configurations across three challenging NETLIB instances, using three method settings and binary choices for pricing, ratio-test, and bound-flip policies [7]. The configurations are not independent observations; this is a policy-robustness stress diagnostic, not Ralph's overall solve rate. Each instance received 24 configurations:

| Stress case | Optimal configs | Recorded outcome |
|---|---|---|
| `pilot.ja` | 0 / 24 | Mostly timeouts; dual settings split between timeout and error. |
| `pilot` | 0 / 24 | Primal settings timed out; dual settings mixed timeout and error. |
| `stair` | 8 / 24 | All optimal runs required bound flipping and dual or automatic method selection. |

In that diagnostic, the fastest successful `stair` configuration required approximately 1.26 times the GLPK wall time. A separate primal-simplex diagnostic timed out on all four pilot-family instances (`pilot.ja`, `pilot`, `pilot.we`, and `pilot87`). These focused, dated repository records do not constitute a comprehensive solver benchmark: they report neither a geometric-mean runtime nor a representative MIP corpus and do not describe performance at the current revision. They support only the narrower conclusion that variation in local control policies had not resolved convergence on the selected difficult LP instances.

**The relevant systems observation is that, on harder models, plausible local improvements did not reliably produce robust whole-solver behavior.** A pivot-rule modification cannot be evaluated solely from the validity of individual pivots; its system-level effect depends on degeneracy, numerical drift, refactorization frequency, and the resulting trajectories across an instance distribution. Presolve, cut generation, branching, primal heuristics, propagation, node selection, conflict analysis, and recovery strategies introduce further interactions that determine whole-search performance. Consequently, feedback about whole-solver quality is slow, noisy, and difficult to attribute because the relevant mechanisms interact through partially hidden dependencies. Directed search along one local dimension can therefore resemble a random walk in the higher-dimensional outcome space: improvement in a local proxy provides weak guidance about global solver quality.

Ralph has modular decomposition and benchmark gates, but these do not make its decisive outcomes locally decidable because the most important interfaces are behavioral and numerical rather than merely syntactic. Sound boundaries can

expose and contain complexity, but they do not remove complexity intrinsic to the domain: faster implementation does not make numerical stabilization or combinatorial search easier. A basis update may satisfy local residual and structural checks yet accumulate damaging numerical error across iterations. A pricing change may reduce iteration count on one subset of instances while increasing degeneracy or numerical instability on another. A mathematically valid cut can still degrade whole-search performance through numerical effects or an unfavorable change to the search tree. A primal heuristic may find valid incumbents yet consume resources better allocated to node processing or other search mechanisms.

Ralph's local decidability could be improved by separating mathematical state, numerical linear algebra, LP control policy, and MIP search policy more explicitly. Candidate invariants include residual bounds, basis-validity checks, monotonic bound and progress accounting, deterministic replay, and explicit recovery contracts. Feedback could be strengthened through stratified benchmark suites, differential feasibility and objective checks, numerical telemetry, controlled ablations, fixed resource budgets, and per-instance regression distributions rather than a single aggregate score.

These measures can improve tractability and assurance, but they do not remove the problem's intrinsic difficulty. Competitive solver behavior depends on decades of accumulated, tightly coupled numerical and algorithmic engineering. Expert architectural judgment can identify missing mechanisms, reject unsound changes, and design better experiments. Yet it cannot reliably predict emergent search behavior or substitute for empirical tuning across heterogeneous instances. Ralph therefore marks a practical boundary: locally successful components and optimal solutions on smaller or easier models do not ensure reliable convergence and performance on larger or harder instances when whole-system behavior is emergent and feedback is costly.

## IV. What determines agentic tractability

The complexity regimes suggest that the engineering value of agent assistance depends less on nominal project difficulty than on the scope and coupling of engineering decisions and the quality and latency of feedback. Keel and Hull are difficult systems, but their explicit contracts and compositional boundaries make many changes locally decidable. Sail raises the integration burden because model, representation, backend, phase, and memory dimensions form a support cross-product. On harder models, Ralph enters an emergent-outcome regime in which local improvements weakly predict whole-solver behavior and feedback is delayed and noisy.

Agent assistance can reduce the implementation-skill bottleneck without necessarily reducing the systems-engineering bottleneck. Broader access to code generation therefore does not imply broader access to the judgment required to engineer reliable complex systems. The systems-engineering expertise that remains necessary includes selecting sound decompositions, identifying consequential invariants, distinguishing representative evidence from local proxies, and recognizing when further implementation is counterproductive.

Decomposition, invariants, and feedback are coupled. Decomposition determines where invariants can be stated; invariants determine what tests must observe; and feedback indicates whether the resulting boundaries provide meaningful isolation or merely organizational convenience. In Sail, separating architecture rules, quantization, placement, and token behavior makes a mismatch easier to attribute. In Ralph, a factorization routine may pass residual checks, or a cut may be mathematically valid, even while whole-solver behavior deteriorates. In such emergent systems, local guarantees remain necessary, but they provide weaker evidence of system-level success.

Architectural judgment is the ability to choose boundaries and constraints under incomplete information. It supports agentic work by narrowing the search space, specifying forbidden behavior, selecting reference behavior, and recognizing when a locally attractive solution undermines system-wide simplicity. A productive loop proceeds iteratively: the engineer defines the frame, the agent explores implementations, tests produce evidence, and the frame is revised when that evidence contradicts its assumptions.

The cases suggest that coding systems can extend an implementation more readily than they can recover the architectural decisions and validation logic that shaped it. Repository contents may preserve implementation artifacts, design rationale, validation evidence, and even rejected alternatives, but a coding system may not reconstruct them as a coherent decision trajectory. Under these conditions, more capable coding systems may amplify a systems expert's impact by allowing that expert to direct substantially greater implementation throughput. This creates a potential asymmetry: implementation throughput may expand faster than the expertise available to evaluate whole-system quality. This yields a testable prediction: as coding agents improve, the productivity differential associated with systems-engineering expertise may initially widen rather than narrow.

Architectural judgment is itself bounded. A high volume of generated changes consumes review attention and can accumulate verification debt faster than an engineer can retire it. Human supervisors may choose an elegant but incorrect decomposition, preserve obsolete assumptions, or overfit validation to known cases. Agreement among coding agents that share models, context, or validation assumptions should not be treated as independent evidence. Architectural judgment can require token parity, authority confinement, or numerical telemetry, but it cannot establish them without representative validation evidence.

Implementation speed introduces a further risk. Early boundaries are hypotheses and should remain cheap to revise. Coding agents can rapidly create downstream consumers of an immature abstraction, increasing the cost of revising or removing it. This is premature architectural hardening: implementation throughput converts uncertainty into dependency before the contracts and feedback regime are stable. Temporarily delaying breadth can therefore increase long-run engineering progress by reducing later rework.

Alternative explanations remain plausible. The cases differ in maturity, expert attention, domain knowledge, reference

implementations, and success criteria. Future coding agents may reason more effectively about long-range interactions. Consequently, the cases do not establish that architectural decomposition caused the more favorable outcomes or that emergent coupling caused the less favorable ones. They support the narrower claim that engineering work with current coding agents is more tractable when correctness is locally decidable, invariants are enforceable, and system-level feedback is timely and representative.

## V. Engineering implications

The practical response is not simply to reduce the use of coding agents, but to direct scarce engineering attention toward architecture and validation. Before increasing implementation throughput, define a provisional decomposition, specify dependency constraints and lifecycle and ownership rules, and establish end-to-end acceptance tests. Treat each generated change as a hypothesis to be tested, and account for review, integration, maintenance, and future explanation as part of its cost.

Maintain an architecture ledger that records responsibility boundaries, forbidden dependencies, invariant ownership, and deliberate scope exclusions. Prefer interfaces that expose semantic operations rather than incidental backend details. Require each optimization proposal to identify its reference path, fallback behavior, affected execution phases, and validation gates. When feedback is sparse, delayed, or difficult to attribute, invest first in observability and representative benchmark design. If representative feedback cannot be obtained at acceptable cost, narrow the supported scope.

The framework also suggests a stopping rule. If implementation creates cross-component assumptions faster than contracts and tests can expose and validate them, additional throughput is increasing verification debt rather than engineering value. At that point, the appropriate response is to pause expansion and consolidate or delete code, or reduce scope, even when generating more code is inexpensive.

## VI. Threats to validity

The four cases come from a single practitioner's work and differ in maturity, domain, and success criteria. The repositories provide observable artifacts and records, but attributing project outcomes to AI assistance or particular architectural choices remains interpretive. No matched versions of these projects developed without coding agents exist. The proposed regimes are analytical categories rather than measured complexity classes, and assigning the four projects to them depends on qualitative judgment. The Ralph table summarizes selected repository diagnostics rather than a representative benchmark, so it cannot establish performance across LP and MIP workloads generally. Claims about Sail's proposed successor architecture describe intended scope rather than demonstrated performance. Documentation written or revised after implementation may incorporate lessons learned later, making the architecture appear more deliberate from the outset than it was.

Future evaluations should compare matched tasks under different architectural specifications, measure review and rework alongside generated volume, and vary feedback latency and coverage. They should also vary the systems-engineering expertise available to direct and evaluate agents, testing whether improved coding capability widens or narrows the resulting productivity differences. Useful outcome measures include escaped defects, invariant violations, cross-module change breadth, performance regressions, and time to diagnose system-level failures.

## VII. Conclusion

In these cases, current coding agents substantially expanded the amount of implementation one engineer could attempt. The cases are consistent with the hypothesis that this added implementation capacity is most valuable when decomposition converts global obligations into locally decidable tasks, invariants are explicit and enforceable, and system-level feedback is timely and representative. Among the cases, Keel provides the strongest retrospective evidence for this hypothesis. Hull offers less mature evidence from an architecture converging under similar principles. Sail provides a prospective test of the framework, while Ralph illustrates both successful optimization on smaller models and the failure of local improvements to ensure reliable convergence on larger, harder ones.

Coding agents therefore tend to amplify the architecture and constraints supplied to them. Architectural judgment supplies this structure by selecting boundaries, exclusions, invariants, and validation requirements. Yet supervisory attention is finite, expert judgments can be mistaken, and architectural judgment cannot substitute for empirical feedback in emergent systems. The practical dividing line for agentic tractability is not whether coding agents can implement components, but whether component-level correctness composes into observable whole-system quality. The cases therefore suggest that the limiting constraints in large AI-assisted software projects may shift away from code production and toward coherent decomposition, architectural judgment, and effective validation. Generating code is becoming cheap. Managing complexity is not.

## VIII. References


[1] artalis.io. "Keel README: architecture and backend axes." github.com/artalis-io/keel/blob/main/README.md, accessed 2026-09-02.
[2] artalis.io. "Keel stream transport contract." github.com/artalis-io/keel/blob/main/docs/contracts/stream.md, accessed 2026-09-02.
[3] artalis.io. "Hull architecture." github.com/artalis-io/hull/blob/main/docs/architecture.md, accessed 2026-09-02.
[4] artalis.io. "Sail architecture roadmap (repository historically named bitnet.c)." github.com/artalis-io/bitnet.c/blob/main/docs/roadmap.md, accessed 2026-09-02.
[5] artalis.io. "Ralph source and benchmark infrastructure." github.com/artalis-io/otto/tree/main/ralph, accessed 2026-09-02.
[6] GNU Project. "GLPK: GNU Linear Programming Kit." gnu.org/software/glpk, accessed 2026-09-02.
[7] artalis.io. "Ralph versus GLPK control and policy convergence record." github.com/artalis-io/otto/blob/main/docs/roadmaps/ralph-vs-glpk.md, accessed 2026-09-02.
[8] F. Gloeckle et al. "Better & Faster Large Language Models via Multi-token Prediction." arXiv:2404.19737, 2024.
[9] J. Chen, Y. Liang, and Z. Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026.
[10] A. Zandieh et al. "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate." arXiv:2504.19874, 2025.
[11] C. E. Jimenez et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" arXiv:2310.06770, 2023.
[12] METR. "Measuring AI Ability to Complete Long Software Tasks." NeurIPS 2025, 2025.
[13] S. Asthana et al. "Runtime-Structured Task Decomposition for Agentic Coding Systems." arXiv:2605.15425, 2026.
[14] T. Santilli, F. Daghero, and M. Tourchi Moghaddam. "SAKE: Software Architectural Knowledge Evaluation Benchmark for Large Language Models." arXiv:2606.29520, 2026.
[15] Y. Zhu, N. Tsantalis, and P. C. Rigby. "AI-Generated Smells: An Analysis of Code and Architecture in LLM and Agent-Driven Development." arXiv:2605.02741, 2026.